CS2100 | Midterm Cheat Sheet

Tien Cheng & Zhao Wu

December 13, 2025

1 C Programming

1.1 Basic Syntax

Variable Assignment and Declaration

- Variable declaration: `int a;`
  - Uninitialized automatic (block-scope, non-static) variables have indeterminate values; reading them is undefined behavior.
  - Objects with static storage duration (file-scope objects and `static` objects) are zero-initialized before program start.
- Assignment: `a = 12;` yields the assigned value (type of the left operand), so:

```
z = (a = 12); // a becomes 12, z becomes 12
```

  - Associates right-to-left: `x = y = z = 42;`
  - Beware: = (assignment) vs == (comparison) in conditions.
- Variable scope: identifiers are scoped to the `{ ... }` block where they are declared. Inner declarations shadow outer ones. Using an out-of-scope name is an error.
- Static variables (in functions): `static int c;` persists across calls.
  - Initialized once; initializer must be a constant expression (or address constant).
  - If no initialization is provided, it's zero-initialized (numeric 0, null pointer).

Control Flow: Switch Statements

- `switch` expression and `case` labels are of integer type (including enum); `case` labels must be integer constant expressions.
- Fall-through by default; use `break` to exit a `case`.

```
switch (expr) {
  case v1: /* ... */ break;
  case v2: /* ... */ /* fall through */
  default: /* ... */ break; // default is optional
}
```

Loops

- `while` checks condition before each iteration:

```
while (condition) { /* body */ }
```

- `do while` runs body once before checking:

```
do { /* body */ } while (condition);
```

- `for` combines init; condition; update (the init's scope is the loop):

```
for (init; condition; update) { /* body */ }
```

- `break` exits the loop (or switch); `continue` skips to next iteration.
- Infinite loop idiom: `for (;;) { /* ... */ }`

1.2 Pointers and Arrays

Pointers

- Pointers hold memory addresses at run time.
- `&var` returns the address of `var`; `T *p;` declares a pointer to `T`; `*p` dereferences `p`.

```
double a, *pb;
pb = &a;        // legal: pb points to a
double c, *pd;
*pd = &c;       // ILLEGAL: *pd is a double lvalue, &c
is double* (type mismatch), and pd is uninitialized
double e, f;
f = &e;         // ILLEGAL: assigning an address to a
non-pointer
double *p = &e; // LEGAL: p points to e
*p = 3.14;      // stores into e via p
```

- Always initialize pointers before use (e.g., to `NULL` or a valid address). Dereferencing an invalid/uninitialized pointer is undefined behavior.
- To print a pointer, use `printf("%p\n", ptr)`, which will print the hex value of the address
- We can have void pointers, `void *ptr` that act as generic pointers (pointing to no particular type, useful for generic functions) but should be type cast to the correct type before use.

Pointer Arithmetic

- If `ptr` is a `T*`, then `ptr + i` points to address `ptr + i * sizeof(T)`.
- Increment/decrement moves by whole elements: `int*` moves by `sizeof(int)`, `char*` moves by `sizeof(char)` (1).
- Subtracting two pointers of the same type gives the element distance: `ptr2 - ptr1` (both must point into the same array, or one past the end).
- To increment the pointed-to value (not the pointer), use parentheses: `(*ptr)++;`

Arrays

- Arrays are contiguous, fixed-size sequences of a single type.
- At declaration, size must be known (statically). Initialization with `{ ... }` is only allowed at declaration.

```
int arr[2] = {1, 2};
int arr2[] = {1, 2, 3};      // size inferred
int some[3] = {1};           // remaining elements
zero-initialized: {1, 0, 0}
// Fill array values with user input
int numbers[3];
for (int i = 0; i < 3; i++) {
  scanf("%d", &numbers[i]);
}
```

- Dynamically allocated array:

```
// include <stdlib.h>
size_t n = 100;
int *dyn = malloc(n * sizeof *dyn); // no cast in C;
checks sizeof via the dereferenced pointer
if (!dyn) { /* handle allocation failure */ }
/* ... use dyn ... */
free(dyn);
```

- We can get the length of an array as follows:

```
int array_length = sizeof(array_name) /
sizeof(array_name[0]);
```

- However, once we pass an array as a function argument, it decays to a pointer to the first element and the `sizeof` operator will instead just return the size of the pointer

Arrays as Pointers

- In most expressions, arrays decay to pointers to their first element; exceptions: `sizeof`, unary &, and string literal to array initialization.
- You cannot assign arrays (`arr1 = arr2;` is illegal), but you can use pointers to elements:

```
int *second = arr2 + 1;  // &arr2[0] + 1
```

- Indexing `arr[i]` is equivalent to `*(&arr[0] + i)`; also `i[arr]` works but is discouraged.

Strings

- Strings are `char` arrays terminated by `'\0'`. Without the terminator, string functions are unsafe.
- The following initializations are equivalent:

```
char str[] = "hello";
char str2[] = {'h','e','l','l','o','\0'};
```

- Iteration patterns:

```
// pointer style (ptr must point to str)
// this works as '\0' is 0 in ASCII, so evaluates to
false
for (char *p = str; *p; ++p) { /* use *p */ }

// index style
for (size_t i = 0; str[i] != '\0'; ++i) { /* use str[i]
*/ }

// avoid recomputing strlen in loop condition
size_t len = strlen(str);
for (size_t i = 0; i < len; ++i) { /* ... */ }
```

- String I/O
  - Input:
    - `fgets(str, size, stdin)`: reads up to `size - 1` chars or newline (keeps newline if space permits).
    - `scanf("%s", str)`: reads until whitespace; use a width to avoid overflow, e.g., `scanf("%99s", str)`.
  - Output:
    - `puts(str)`: writes `str` then newline.
    - `printf("%s\n", str)`: writes until `'\0'`.
- Common string functions:
  - `strlen(s)`: number of chars before `'\0'` (terminator not counted).
  - `strcmp(s1, s2)`: returns 0 if equal, < 0 if `s1 < s2`, >0 otherwise (lexicographic by unsigned char values).
    - Bounded: `strncmp(s1, s2, n)`.
  - `strcpy(dst, src)`: copies `src` into `dst` (requires `dst` large enough).
    - Bounded: `strncpy(dst, src, n)` but beware: it does not always append `'\0'` if truncated.
    - Safer patterns: ensure capacity and terminate manually, or use `strlcpy` where available (non-standard).

1.3 Structs and Unions

Structs

- A `struct` groups related fields into one object.

```
struct Point {
  int x;
  int y;
};

struct Point p1 = { .x = 10, .y = 20 };   // designated
init
struct Point p2 = { 1, 2 };               // positional
init
```

- `typedef` to avoid repeating `struct`:

```
typedef struct {
  int x, y;
} Point;

Point p = { .x = 3, .y = 4 };
```

- Passing/returning structs:

```
Point make_point(int x, int y) { return (Point){ x,
y }; } // compound literal
void move(Point *p, int dx, int dy) { p->x += dx; p->y
+= dy; } // pass pointer to modify
```

- Memory allocation: struct members are stored in memory in the order they are declared within the struct definition. Note that if you declare a `char*`, the memory allocated will be only for the pointer, the string itself will be stored somewhere else in memory

Partial Initialization

- It is possible to partially initialize a struct; any members not explicitly initialized are zero-initialized.
  - Positional initialization fills fields in order; missing trailing fields become zero.
  - You cannot skip a middle field without designators; use designated initializers to name only the members you want.

```
typedef struct { int a; double b; int c; } S;

S s1 = { 42 };                  // a=42, b=0.0, c=0
S s2 = { 1, 2.5 };              // a=1,  b=2.5, c=0
S s3 = { .c = 7 };              // a=0,  b=0.0, c=7
(designated init)
S s4 = { .b = 3.14, .a = 9 };  // order of designators
can vary; unspecified become zero
```

Arrow vs Dot Operator

- `.` accesses a member of a struct object.
- `->` accesses a member through a pointer to struct (equivalent to `(*ptr).member`).

```
Point p = {1, 2};
Point *pp = &p;

int a = p.x;      // dot
int b = pp->y;    // arrow, same as (*pp).y
```

- Parentheses matter when combining with `*`:

```
*pp.x      // ERROR: '.' binds before '*'; tries to
access field on pp (not a struct)
(*pp).x    // OK
pp->x      // OK (preferred)
```

Nested Structs and Arrays/Strings

```
typedef struct {
  char name[32];
  Point pos;
} Sprite;
```

```
Sprite s = { .name = "player", .pos = { .x = 10, .y =
20 } };
```

1.4 Functions and Parameter Passing

Pass-by-Value vs 'By-Reference'

- C passes arguments by value only: a function receives copies of the arguments.
- To let a function modify a caller's object, pass the object's address (pointer). This is often called "pass-by-reference" idiomatically.

```c
// Pass-by-value: does not change caller's x and y
void swap_bad(int x, int y) {
  int t = x; x = y; y = t;
}

// Pass 'by reference': pass addresses so the function
can modify caller's ints
void swap(int *x, int *y) {
  int t = *x; *x = *y; *y = t;
}

int a = 1, b = 2;
swap_bad(a, b);     // a==1, b==2
swap(&a, &b);       // a==2, b==1
```

- Arrays in parameter lists decay to pointers; the function receives a pointer to the first element, not a full array copy:

```c
void sum(const int *arr, size_t n, long long *out) {
  long long s = 0;
  for (size_t i = 0; i < n; ++i) s += arr[i];
  *out = s;
}
```

- Use `const` to signal "this pointer points to data I will not modify": `void f(const int *p);`
- Distinguish:
  - `const int *p` — pointer to const int (you cannot modify `*p` via `p`).
  - `int * const p` — const pointer to int (the pointer value can't change, but `*p` can).
  - `const int * const p` — both are const.

Passing Structs to Functions

- When structs are passed into a function, we actually perform a pass by value, meaning that we copy all the values within the struct
- This includes arrays, so modifying the array inside a struct within a function does not affect the original array
- Exception: if the data type is defined as a pointer (e.g using `char *` instead of `char[]`) then the pointer copied over still references the original array

Returning multiple values

- Return one value directly; return others via pointer parameters or structs:

```c
typedef struct { int min, max; } MinMax;
MinMax bounds(const int *a, size_t n);
```

2 Data Representation

Data Representation

In C, a data type of a variable determines its interpretation from binary.

Type	Bits	Min Values	Max Values
_Bool or bool (if `#include <stdbool.h>`)	8	0	1
char	8	0	2^8 ; 255
short	16	-2^{15} ; -32768	$2^{15}-1$; 32767
unsigned short	16	0	$2^{16}-1$; 65535
int	32	-2^{31} ; -2147483648	$2^{31}-1$; 2147483647
unsigned int	32	0	$2^{\{32\}}-1$; 4294967295
long	64	-2^{63}	$2^{63}-1$
unsigned long	64	0	$2^{64}-1$
float	32	$-3.4028235 \times 10^{38}$	3.4028235×10^{38}
double	64	-1.79769×10^{308}	1.79769×10^{308}

```c
int number = 70; // 0b01000110

printf("Represented as Digit %d\n", number); // 70
printf("Represented as Char %c\n", number); // F
```

Common Bytes Quantifier

- Byte: 8 bits
- Nibble: 4 bits
- Word: Architecture-dependent fixed width (MIPS uses 1 word = 32 bits = 4 bytes)

Number of Represented Values

- N bits represent up to 2^N values
- $\lceil \log_2 M \rceil$ bits represents M values

IEEE 754 Hex → Decimal (Single Precision)

- A 32-bit float uses: 1 sign bit, 8 exponent bits (bias 127), 23 fraction bits with an implicit leading 1 for normalized numbers.
- Exam recipe to convert hex→decimal:
  1. Convert the 32-bit hex to binary; split into sign `s`, exponent `E`, fraction `F`.
  2. Compute sign: $(-1)^s$ and unbiased exponent: $e = E - 127$ (for $0 < E < 255$).
  3. Build mantissa: $M = 1.F$ in binary (hidden leading 1) for normalized values, or $M = 0.F$ if $E = 0$ (denormals).
  4. Value is $x = (-1)^s \times M \times 2^e$; special cases: $E = 255, F = 0 \to \pm\infty$; $E = 255, F \neq 0 \to$ NaN; $E = 0, F = 0 \to \pm 0$.
- For decimal→hex, reverse the process: choose the sign, normalize the value to $1.F \times 2^e$, then store the biased exponent $E = e + 127$ and the lower 23 bits of the fraction field.

3 Number System

3.1 Weighted Positional Number System

- In a **weighted-positional number system** of base r each position holds *multiples* of *power of radix*.
  - Can have *radix* number of unique symbols.
  - E.g. Decimal number system is a **weighted-positional** number system, where the Base (aka Radix) is 10.

$$(a_n a_{n-1} l \ldots a_0 . f_1 f_2 l \ldots f_m)_r = (a_n \times r^n) + (a_{n-1} \times r^{n-1}) + l \ldots + (a_0 \times r^0) + (f_1 \times r^-1) + (f_2 \times r^-2) + l \ldots + (f_m \times r^- m)$$

Convert Base-R to Decimal

- $(1101.101)_2 = 2^3 + 2^2 + 2^0 + 2^{-1} + 2^{-3} = 13.625_{10}$
- $(341.24)_5 = 3 \times 5^2 + 4 \times 5^1 + 1 \times 5^0 + 2 \times 5^{-1} + 4 \times 5^{-2} = 96.56_{10}$

Convert Decimal to Binary / Base-R

- For **Whole Number**, press calculator if you can. Or compute manually through *division-by-base* method (when dealing with non-conventional bases).

2	42
2	21 rem **0** ⇐ LSB
2	10 rem **1**
2	5 rem **0**
2	2 rem **1**
2	1 rem **0**
0	rem **1** ⇐ MSB

$$(42)_{10} = (101010)_2$$

- For **Fractions**, use *multiplication-by-base* method.

	Carry
$0.3125 \times 2 = 0.625$	**0** ⇐ MSB
$0.625 \times 2 = 1.25$	**1**
$0.25 \times 2 = 0.5$	**0**
$0.5 \times 2 = 1$	**1** ⇐ LSB

$$(0.3125)_{10} = (0.0101)_2$$

- When we cannot fully represent a number in the desired number of fractional bits, we should calculate up to the number of bits we want + 1, then check the LSB to determine if we should round up
- We may **convert fraction using calculator** by multiplying by a power of smallest exponent, convert, then divide back.

$$(0.0101)_2 = (101)_2 * 2^{-4} = \frac{(5)_{10}}{2^4} = 0.3125$$

Binary to Octal / Hexadecimal Conversion

- Binary to Octal: Partition in group of 3. Vice-versa.

$$(10\ 111\ 011\ 001.101\ 110)_2 = (2731.56)_8$$

- Binary to Hexadecimal: Partition in group of 4. Vice-versa.

$$(101\ 1101\ 1001.1011\ 1000)_2 = (5D9.B8)_{16}$$

Base R1 to Base R2 Conversion

- In general, for a given base R we can convert to base R^n quickly with the shortcut of grouping into groups of size n. For example to convert from base 3 to base 9, we can group the digits into groups of 2 since $9 = 3^2$
- For other possible base conversion, we may first convert to decimal, before converting to targeted base.

3.2 ASCII

- ASCII encodes characters using **7 information bits**; some transports add an optional parity bit as the MSB, but it is not part of the code itself.
  - Even Parity: `number_of_1s == even ? 0 : 1`
  - Odd Parity: `number_of_1s == odd ? 0 : 1`
- Convert between lower and uppercase thru addition/subtraction of 32.

```c
char uppercase = 'A';
char lowercase = uppercase + 32;  // Add 32 to convert
upper to lower
```

Even and Odd Bit Parity Scheme

- Append 1 parity bit so that total 1-bits (data + parity) is even (even parity) or odd (odd parity).
- Let `p_even = d0 ^ d1 ^ ... ^ d(n-1)` over the data bits; then `p_odd = !p_even & 1`.
  - Even parity uses `p = p_even`; odd parity uses `p = p_odd`.
- Example (7-bit data): `1010001` has three 1s →
  - Even parity `p = 1` to make 4 ones total.
  - Odd parity `p = 0` to keep 3 ones total.
- Placement (MSB/LSB) is protocol-dependent; classic ASCII links often used MSB as parity.
- Detects any single-bit error by recomputing parity; does not correct errors and can miss some multi-bit flips.

3.3 Negative Numbers

There are 4 common ways to represent negative numbers:

1. **Sign-and-Magnitude**
   - **MSB Sign bit**, `0 = Positive, 1 = Negative`.
   - Fixed number of bits as **magnitude**.
   - **Negate** by flipping sign bit.
   - **Range**: $[-(2^{n-1}-1), 2^{n-1}-1]$
   - **Problems**:
     - redundant prefix 0s for small number
     - distinguish between +0 and −0
     - inability to represent large number as limited by number of magnitude bits.
2. **1s Complement**

- Aka **Diminished Radix Complement**:
  - 1s complement is the diminished radix-2 complement: $(x)_{1s} = (2^n - 1) - x$
  - Or more formally, a diminished radix complement for radix/base = b of N-bit whole number, M-bit fraction can be represented as:

    $$-x = b^n - x - b^m$$
- **Negate** by:
  - Flipping all the bits
  - To find the decimal representation of negative number $-x = 2^n - x - 1$
    e.g. $-12 = 2^8 - 12 - 1 = 243 = (11110011)_2$
- **Range**: $[-(2^{n-1} - 1), 2^{n-1} - 1]$
- **Arithmetic**:
  - Bitwise addition. If there is a carry out of the MSB, wrap the carry around and add it to the LSB.
  - Example (8-bit):

    $7 - 3 = (00000111)_2 + (11111100)_2 = (00000011)_2$ with carry 1 End-around: $(00000011)_2 + 1 = (00000100)_2 = 4$
- **Problems**:
  - distinguish between +0 and −0

3. **2s Complement**
   - Aka **Radix Complement**:
     - 2s complement is the radix-2 complement: $(x)_{2s} = (2^n) - x$
     - Or more formally, a radix complement for radix/base = b of N-bit can be represented as:

       $$-x = b^n - x$$
   - **Negate** by:
     - Flipping all the bits and +1.
     - Faster way: from right to left, copy all the bits until the first 1, copy over the 1, then from then on, flip the bits
     - To find the decimal representation of negative number $-x = 2^n - x$
       e.g. $-12 = 2^8 - 12 = 244 = (11110100)_2$
     - Faster way: The MSB will have weight of -2^{n-1}
   - **Range**: $[-2^{n-1}, 2^{n-1} - 1]$
   - **Arithmetic** is the same as 1s Complement, with the exception that we ignore any carry-out from the MSB
4. **Excess**
   - Represent first N as negatives in Excess-N on M-bit.
   - Value 0 is **encoded** by $0 + N = C$, **decoded** by $C - N = 0$
   - Easy comparison by looking at bit-values alone.
   - **Negate** by:
     1. In Excess-127 system, encode word for (x = 5):

        $$C = x + N = 5 + 127 = 132$$
     2. Negated code:

        $$2N - C = 2(127) - 132 = 122$$
     3. Decode 122:

        $$x = C - N = 122 - 127 = -5 = -x$$
   - **Range**:
     - For even distribution between +ve and -ve numbers, we compute $N = 2^{n-1}$
       - range= $[-2^{n-1}, 2^{n-1} - 1]$
     - For non-even distribution, range = $[-N, 2^n - N - 1]$
   - **Arithmetic**:
     - Bit-wise addition no longer works.
     - Decode to base number, perform arithmetic, convert back.

Negative Numbers Shenanigans

- Overflow can occur when adding two positive numbers or two negative numbers
  - Positive + Positive = Negative
  - Negative + Negative = Positive
- This is due to the result being unable to be represented in the given bit width
- Overflow can be detected by checking if the sign bit is consistent with the operators. Note that overflow cannot happen when adding two numbers of opposite sign.
- In general, for a 32-bit signed integer represented via two's complement, overflow happens once we exceed the max or min range $[-2^{31}, 2^{31} - 1]$, in which case we expect the value to wrap around

3.4 Real Numbers

There are 2 ways to represent real number:

1. **Fixed Point Representation**
   - **Fixed number of bits** for *whole* number and *fraction*.
   - Extension of 1s Complement or 2s Complement.
     - **Negate** in 2s Complement by inverting all bits and adding one unit in the last place (value $2^{-\#\text{fraction bits}}$)
   - **Approximation** of real number as could not represent number smaller than $2^{-\#\text{fraction bits}}$
   - **Round up the fraction bit** if fraction overflow occurs.
     - E.g. Given 3-bit fraction: $(0.1001)_2 \Rightarrow (0.101)_2$
2. **Floating Point Representation**
   - IEEE 754 floating-point representation
     - 1-Bit **Sign bit** (`is_positive ? 0 : 1`)
     - 8-Bit **Exponent** (Excess-127)

       $$\text{bin}((\text{exponent} + 127)_{10})_2$$
     - 23-Bit **Mantissa** (normalized with implicit leading bit 1)

       $$\text{sign} \quad 1.\text{mantissa} \times \text{base}^{\text{exponent}}$$
       $$(6.5)_{10} = (110.1)_2 = (1.101) \times 2^2;$$
       $$\text{sign} = 0, \quad \text{exponent} = (2)_{10}, \quad \text{mantissa} = (101)_2$$
   - In 32-bit Floating Point (Single-precision)

(1-bit) Sign	(8-bit) Exponent <excess-127>	(23-bit) Mantissa

   - In 64-bit Floating Point (Double-precision)

(1-bit) Sign	(11-bit) Exponent <excess-1023>	(52-bit) Mantissa

Real Numbers Shenanigans

- Associativity is not preserved in FP arithmetic as rounding occurs.

```
(A op B) op C = round(round(A op B) op C)
```

- Ranges of Numbers in FP

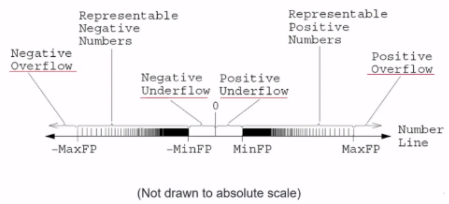


Figure 1: FP Range

4 Instruction Set Architecture (ISA)

- ISA defines the contract between Hardware and the instructions.
- Same instructions should result in same behavior, regardless of hardware implementation.

CISC vs RISC

There are two general types of ISA design.

- **Complex Instruction Set Computer** (CISC)
  - Smaller program size
  - Complex processor implementation, difficult hardware optimization
  - E.g. x86-32, IA32
- **Reduced Instruction Set Computer** (RISC)
  - Small and simple instruction set, easier to optimise for hardware
  - Burden on software to compile efficiently
  - E.g. MIPS, ARM

Data Storage

Some major storage architecture in processor includes:

1. **Stack Architecture**
   - **Implicit** operands on top of the stack.
   - `Push A; Push B; Add; Pop C`
2. **Accumulator Architecture**
   - **Implicit** operands as stored in the Accumulator (a special register)
   - `Load A; Add B; Store C`
   - E.g. IBM 701, DEC PDP-8
3. **General-Purpose Register Architecture** (GPR)
   1. **Register-Memory** Architecture
      - **Explicit** Operand. **One operand** can be in **memory**.
      - E.g. Motorola 68000, Intel 80386
      - Typically found in **CISC**.
   2. **Register-Register** Architecture (Load/Store Architecture)
      - **Explicit** Operand. **All operand** must be in **register**.
      - `Load R1, A; Load R2, B; Add R3,R2,R1; Store R3, C;`
      - Typically found in **RISC**.
      - E.g. MIPS, DEC Alpha
4. **Memory-Memory** Architecture
   - **Explicit** Operand. **All operand** in **memory**.
   - `Add C, A, B`
   - Typically found in **CISC**.
   - E.g. DEC VAX

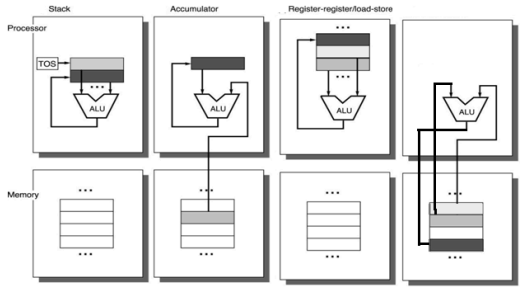


Figure 2: Data Storage in Processor

4.1 Memory Addressing mode

- Given k-bit address, the address space is 2^k.
- Each memory transfer consist of one word of n-bit.

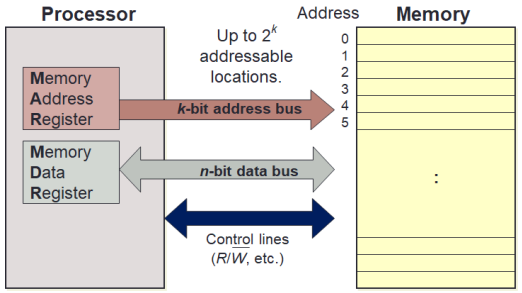


Figure 3: Memory Bus

Endianness

- Determine relative ordering of bytes in multiple-byte word stored in memory.
- Does not affect the absolute order of different words.
- Does not affect relative order of bytes in a single address.

	Big-Endian	Little-Endian
Example	`0x DE AD BE EF` is stored as Figure 4: Big Endian	`0x DE AD BE EF` is stored as Figure 5: Little Endian
Description	MSB is stored in lowest address.	LSB is stored in lowest address.

E.g.	IBM 360/370, Motorola 6800, MIPS, SPARC	Intel 80x86, DEC VAX, DEC Alpha

Addressing Mode

Addressing Mode	Example	Meaning
Register	`add $t0, $t1, $t2`	$\$t0 \Longleftarrow \$t1 + \$t2$
Immediate	`addi $t0, $t0, 3`	$\$t0 \Longleftarrow \$t0 + 3$
Base + Offset	`lw $t0, 100($s1)`	$\$t0 \Longleftarrow M[100 + \$s1]$
Register Indirect	`lw $t0, ($s1)`	$\$t0 \Longleftarrow M[\$s1]$
Indexed	`lw $t0, ($s1 + $t2)`	$\$t0 \Longleftarrow M[\$s1 + \$t2]$
Absolute	`ADD R1, (1001)`	$R1 \Longleftarrow R1 + M[1001]$
Memory Indirect	`ADD R1, @R3`	$R1 \Longleftarrow R1 + M[M[R3]]$
Auto-Increment	`ADD R1, (R2)+`	$R1 \Longleftarrow R1 + M[R2]$; $R2 \Longleftarrow R2 + d$
Auto-Decrement	`ADD R1, -(R2)`	$R2 \Longleftarrow R2 - d$; $R1 \Longleftarrow R1 + M[R2$
Scaled	`ADD R1, 100(R2)[R3]`	$R1 \Longleftarrow R1 + M[100 + R2 + R3 \times d]$

4.2 Operations in ISA

Operations	Examples
Data Movement	load (*from memory*) store (*to memory*) memory-to-memory move register-to-register move input (*from I/O device*) output (*to I/O device*) push (*to stack*) pop (_from stack)
Arithmetic	*the operations typically applicable to both integer and floating point* add subtract multiply divide
Shift	shift left shift right rotate left rotate right
Logical	not and or set clear
Control Flow	jump (*unconditional*) branch (*conditional*)
Subroutine Linkage	call (*to procedure*) return (*from procedure*)
Interrupt	trap return
Synchronisation	test & set (*atomic r-m-w*)
String	search move compare
Graphics	pixel and vertex operations compression/decompression

Figure 6: Standard ISA Operations

4.3 Instruction Formats

Two type of instruction formats:

1. Variable-Length Instruction
   - Single instruction can have different size
   - Require multi-step fetch and decode
   - Allow for more flexible (but complex) compact instruction set.
2. Fixed-Length Instruction
   - Every instruction have fixed size
   - Allow for easy fetch and decode
   - Simplify pipelining and parallelism

Instruction Fields

- An instruction consist of:
  - `Opcode`: Unique code to specify desired operation
  - `Operands`: Zero or more additional information needed for the operation.
- Typical type and size are as follows:
  - Characters: 8 bits. (1-byte)
  - Half-Word: 16 bits. (2-byte)
  - Word: 32 bits. (4-byte)
  - Single-Precision Floating Point: 1 word.
  - Double-Precision Floating Point: 2 words.

4.4 Encoding of Instruction Set

- To encode an instruction, we need to include an opcode, and zero or more operands.
- Number of bits allocated to the opcode determines total number of instruction available in the ISA.
- There are generally two different ways to store opcode:
  1. Fixed Length Opcode
     - Inefficient as lead to unused opcode.
     - Number of instructions = $2^{\#\text{opcode_bits}} = 2^6$

Types	6 Bits	5 Bits	5 Bits
A	opcode	Operand	Operand
B	opcode	Operand	unused

  2. Expanding Opcode
     - Address the issue with Fixed Length Opcode by utilizing all the encoding spaces.

Types	6 Bits	5 Bits	5 Bits
A	opcode	Operand	Operand
B	opcode	Operand	opcode1

     - This, however, introduce ambiguity on how should we allocate opcode space in first 6-bit to Types A and Types B to result in different number of total instructions.

Intuition

- To achieve **maximum** number of instructions, we need to minimize `opcode` in A, to get more encoding space in B, that can be multiplied with `opcode1`.
  - Type-A: `opcode=000000`
  - Type-B: `opcode=000001-111111 * opcode2=00000-11111`
    $\#\text{Instructions} = a + b = 1 + (2^6 - 1 * 2^5) = 2017$
- To achieve **minimum** number of instructions, we need to maximise `opcode` in A, and only allocate one `opcode` space for B.
  - Type-A: `opcode=000000-111110`
  - Type-B: `opcode=111111 * opcode2=00000-11111`
    $\#\text{Instructions} = a + b = (2^6 - 1) + 2^5 = 95$

Formula Approach

- Let a, b, c be number of instructions for Type A (6-opcode), Type B (8-opcode), Type C (11-opcode).

$$\frac{a}{2^6} + \frac{b}{2^8} + \frac{c}{2^{11}} = 1$$
$$2^{11-6=5}a + 2^{11-8=3}b + c = 2^{11}$$
$$2^5a + 2^3b + c = 2^{11}$$

- To obtain minimum number of instructions, we will maximise type with highest coefficient (i.e. a), and set the rest as 1 first.

$$2^5a + 2^3(1) + 1 = 2^{11}$$
$$a = \frac{2039}{2^5} \approx 63.718 = 63$$
$$2^5(63) + 2^3(b) + c = 2^{11}; 8b + c = 32;$$
$$8(3) + (8) = 32;\ \min(\text{b}) \text{ while c} > 1$$
$$\therefore a = 63; b = 3; c = 8; \#\text{instructions} = 63 + 3 + 8 = 74$$

- To obtain maximum number of instructions, we will maximise type with lowest coefficient (i.e. c), and set the rest as 1 first.

$$2^5(1) + 2^3(1) + c = 2^{11}$$
$$c = 2^{11} - 40 = 2008$$
$$\therefore a = 1; b = 1; c = 2008; \#\text{instructions} = 1 + 1 + 2008 = 2010$$

5 MIPS Instruction

R-format

- Fields:

opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)

- rs is set to 0 for `sll` and `srl` instructions
- shamt is set to 0 for all non-shift instructions
- `shamt` is 5 bits because MIPS shifts only use the lower 5 bits of the amount, so immediates (and `sllv/srlv/srav`) can shift 0–31 positions on a 32-bit word
- All R-format instructions have an opcode of 0, R-type instructions distinguished by funct

I-format

- Fields:

opcode (6)	rs (5)	rt (5)	immediate (16)

- **Arithmetic I-Type** (e.g addi, slti): immediate represents 16-bit 2's complement
- **Load/Store I-Type** (e.g lw, sw): immediate represents 16-bit 2's complement used as offset from base address `$rs`
- **Logical I-Type** (e.g andi, ori): immediate represents 16-bit raw unsigned binary, zero-extended to 32 bits
- **Branch Instructions** (e.g beq, bne): the immediate value represents the displacement relative to the PC
  - PC-relative addressing:
    - If taken: PC = PC + 4 + (immediate × 4).
    - If not taken: PC = PC + 4.
  - PC-relative addressing can branch $\pm 2^{15}$ words ($\pm 2^{17}$ bytes) from the PC because the 16-bit immediate is sign-extended, shifted left by two, and added to `PC + 4`.
    - If the hardware did not perform that implicit shift, a raw 16-bit value would only give $\pm 2^{13}$ words because instructions are word-aligned.

- In MIPS the shift always happens for you, so the assembler lets you write the offset in words.
  - Exam tip for encoding: for a branch at address `PC` targeting label at `Target`, the stored immediate is `imm = (Target - (PC + 4)) / 4` (expressed in 16-bit 2's complement).

J-format

- Fields

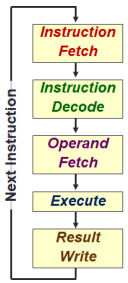
opcode (6)	address (26)

- MIPS will take the 4 MSB from PC + 4, omit 2 LSB since instruction addresses are word aligned (a word is 2^2 bytes)
- Maximum Jump Range = current 2^{28} bytes = 256MB region
  - An address is outside of the region if the 4 MSB of the address do not match the MSB of PC + 4
- To jump further, either use `jr` (jump to address stored in register), or do a `beq $0, $0, L` to jump to the region you want to jump to as intermediate
- Exam tip for encoding: the 26-bit jump field stores bits [27:2] of the target address; the actual target is formed by concatenating the high 4 bits of `PC + 4`, the 26-bit field, and two 0 LSBs.

6 MIPS Datapath

The basic instruction cycle consists of the following five steps:

1. Fetch
   - Get instruction from memory.
   - Address stored in the $PC register (i.e., program counter).
2. Decode & Operand Fetch
   - Find out the operation required.
   - Get the operand(s) needed for operation.
3. ALU
   - Execute arithmetic and logical operations on ALU unit.
4. Memory
   - Read/write from memory.
5. Writeback (a.k.a. Result Write)
   - Store the result of the operation.



Stage	add $3, $1, $2	lw $3, 20($1)	beq $1, $2, label
Fetch	standard	standard	standard
Decode	standard	standard	standard
Operand Fetch	Read [$1] as opr1 Read [$2] as opr2	Read [$1] as opr1 Use 20 as opr2	Read [$1] as opr1 Read [$2] as opr2
Execute	result = opr1 + opr2	addr = opr1 + opr2 Use addr to read memory	taken = (opr1 == opr2) target = (PC + 4) + (label × 4)
Writeback	[$3] = result	[$3] = mem data	if (taken) then PC = target

6.1 Critical Path

Calculating Critical Path

- Critical Path: path taken by slowest instruction through the datapath
- The total delay of the critical path, which is the sum of the latencies of all the components along the path, determines the minimum clock period T_c, and hence the maximum clock frequency $\frac{1}{T_c}$

Load Instruction Critical Path

- Load instructions are usually the critical path in a single-cycle design as it sequentially uses the five major hardware units and has to access memory twice (instruction + data) and write to register file within a single cycle

Typical Critical Path:
1. Instruction Fetch: PC -> Instruction Memory
2. Register Read: -> Register File
3. ALU Operation: -> ALU (address calculation)
4. Memory Access: -> Data Memory (read data)
5. Write Back: -> MUX (MemToReg) -> Register File (write data)

Store Instruction Critical Path

- Store instructions are similar to load instructions but do not include the final write-back to the register, and thus are typically faster to execute Typical Critical Path:

1. Instruction Fetch: PC -> Instruction Memory
2. Register Read: -> Register File
3. ALU Operation: -> ALU (address calculation)
4. Memory Access: -> Data Memory (write data)

R-type Instruction Critical Path

- R-type instructions do not access the data memory

Typical Critical Path:
1. Instruction Fetch: PC -> Instruction Memory
2. Register Read: -> Register File -> MUX (ALUSrc)
3. ALU Operation: -> ALU
4. Write Back: -> MUX(MemToReg) -> Register File (write result)

Branch Instruction Critical Path

- Branch instructions calculates a target address and updates the PC if the condition is met. It does not write to the register file or data memory

Typical Critical Path:
1. Instruction Fetch: PC -> Instruction Memory
2. Register Read: -> Register File -> MUX (ALUSrc)
3. ALU Operation: -> ALU (subtract register to check for zero)
4. PC Update: -> AND Gate (Branch & isZero) -> MUX (PCSrc)

7 MIPS Control Path

7.1 MIPS Control Signals

1. RegDst (MUX Before Register File):
   - Select between `rt` [0] and `rd`[1]. Only select rd in R-instructions.
2. RegWrite (Register File):
   - Determine to write data in `WD` to `WR` [1]
3. ALUSrc (MUX Before ALU):
   - Select between data from `[rt]`(R-instructions) [0] or arithmetic-signed-extended Immediate values (I-instructions) [1].
4. ALUControl (ALU)
   - 4-bit control signal to determine what operation ALU should perform
5. PCSrc (MUX for PC Counter)
   - Determine to increment `PC + 4` [0] or `PC + 4 + instructions * 4` [1] (Branch-Instructions) ⟹ If isZero (from ALU) is True
6. MemWrite (Data Memory)
   - Asserted for store instructions to write data to memory (stores): [1]
7. MemRead (Data Memory)
   - Asserted for load instructions to read data from memory (loads): [1]
8. MemToReg (MUX after Data Memory)
   - Determine to parse ALU Result (R-Instructions) [0] or Memory Read (Read Instructions) result back to WD [1]

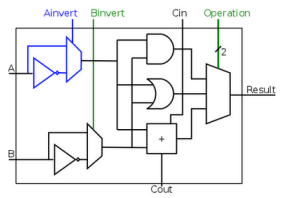
7.2 ALU Control Signal

- 4-Bit control signal to determine how ALU should act according to different function.

ALUControl	Function
0000	and
0001	or
0010	add
0110	sub
0111	slt
1100	nor

1-Bit ALU

- An ALU is a stack of multiple 1-Bit ALU.

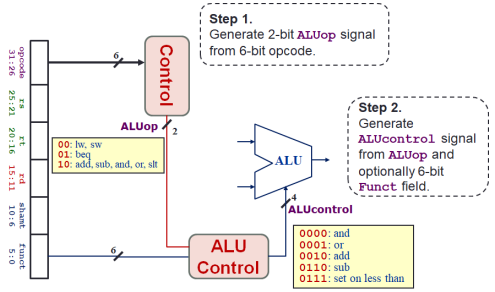


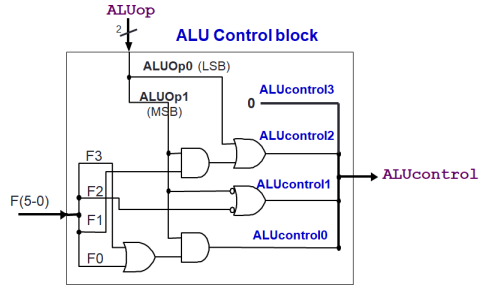
There are 4 control bits needed:

- `Ainvert` (*1-bit*)
  - `0`: Do not invert input `A`
  - `1`: Invert input `A`
- `Binvert` (*1-bit*)
  - `0`: Do not invert input `B`
  - `1`: Invert input `B`
- `Operation` (*2-bits*)
  - `00`: Result of bitwise AND
  - `01`: Result of bitwise OR
  - `10`: Result of ADD

Multilevel Decoding

- Intuitively, `ALUControl (4-bit)` depends on both `opcode` and `funct` field. But instead of generating all 4-bits directly, MIPS takes a multilevel decoding approach.
  - `Control(opcode) = ALUop (2-bit)`
  - `ALUControlBlock(ALUop) = ALUControl (4-bit)`






```
ALUcontrol3 = 0
ALUcontrol2 = (F1 . ALUop1) + (ALUop0)
ALUcontrol1 = !ALUop1 + !F2
ALUcontrol0 = (F0 + F3) . ALUop1
```

	ALUop	Funct	ALUcontrol
lw	0 0	X X X X X X	0 0 1 0
sw	0 0	X X X X X X	0 0 1 0

beq	0 1	X X X X X X	0 1 1 0
add	1 0	1 0 0 0 0 0	0 0 1 0
sub	1 0	1 0 0 0 1 0	0 1 1 0
and	1 0	1 0 0 1 0 0	0 0 0 0
or	1 0	1 0 0 1 0 1	0 0 0 1
slt	1 0	1 0 1 0 1 0	0 1 1 1

7.3 Control Signal In a Nutshell

Control Unit turn opcode to control signals as shown in the following table.

Instruction	opcode	Ctrl
R-Format	`0 0 0 0 0 0`	`1 0 0 1 0 0 0 1 0`
lw	`1 0 0 0 1 1`	`0 1 1 1 1 0 0 0 0`
sw	`1 0 1 0 1 1`	`X 1 X 0 0 1 0 0 0`
beq	`0 0 0 1 0 0`	`X 0 X 0 0 0 1 0 1`

Ctrl	Signal	Selector
0	RegDst	R-Format
1	ALUSrc	lw OR sw
2	MemToReg	lw
3	RegWrite	R-Format OR lw
4	MemRead	lw
5	MemWrite	sw
6	Branch	beq
7	ALUop1	R-Format
8	ALUop0	beq

See reference for more comprehensive table.

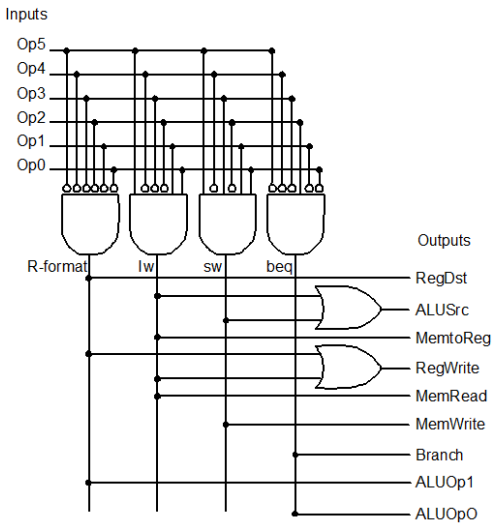


Figure 11: AND gate with collapse NOT gate.
If `000000`, R-Format = 1, If `100011, lw=1,...`

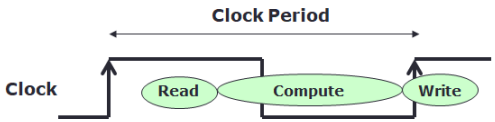
Exam Tip: Reasoning About Datapath Faults

- When a control signal is "stuck" high or low, compare which instruction classes misbehave by reading this table:
  - Example: if R-format and `sw` behave correctly but `addi`/`lw` write the wrong registers, suspect `RegDst` stuck at 1 – the datapath is always selecting `rd` instead of `rt`, which breaks I-type writes but not R-type.
- More generally, group instructions by which control bits they share; if "all instructions that need X fail, but those that don't use X work", X is a good candidate for the faulty signal.

7.4 Multicycle & Pipelining

Single-Cycle

In single-cycle implementation, everything stage happens within a clock period. Which means clock period depends on the maximum time of instruction completion.



Multi-cycle Implementation

- A multi-cycle implementation breaks up instruction execution into steps (Instruction Fetch, Instruction Decode & Operand Fetch, Execute, Memory and Writeback)
- Typically we just break up into 1 step for each stage, so a instruction can take up to 5 execution steps, where each step is one clock cycle
- Advantage: each clock cycle is smaller since it can just run at the speed of the slowest stage which is still better than the sum of all the stages, hence for instructions that don't actually require all of the stages (e.g no memory access or no-write back), the instruction will execute faster.
- If an instruction doesn't need every stage, multi-cycle is faster since we avoid redundant time waiting for irrelevant stages to execute. A single cycle implementation must support the slowest instruction, leading to a long clock cycle for all instructions
- If an instruction takes every stage:
  - and each stage takes same amount of time to complete: then execution time is same as single-cycle
  - but if some stages take longer, the clock cycle is set by the slowest stage, so the instruction could actually be slower than single-cycle implementation
- Thus, to determine the clock speed:
  - Single Cycle: find latency of the slowest instruction
  - Multi Cycle: find latency of the slowest stage
  - Clock Speed = 1 / Latency